

Linear Regression & Optimization

From Scratch

Mathematical derivations, Python implementations, experiments, and
visualization gallery

Dataset	Hours studied vs. exam score
Core tools	Python, NumPy, Matplotlib
Optimization	Gradient Descent, Momentum, Nesterov, Ada-Grad, RMSProp, Adam
Analysis	Loss surfaces, gradients, Hessian, eigenvalues, stability, scaling
Project folder	01_linear_regression

Complete technical study and experiment record

Contents

1 Project Overview	2
2 Dataset and Model	2
3 Mean Squared Error	2
4 Gradient: Full Derivation	2
4.1 Derivative with respect to the intercept	3
4.2 Derivative with respect to the slope	3
5 Gradient Descent	3
6 Loss Curve and Convergence	3
7 Loss Surface and Geometry	4
8 Gradient Direction and a First-Order Check	4
9 Gradient Checking	4
10 Learning Rate	4
11 Hessian, Eigenvalues, and Stability	4
12 Stability Experiments	5
13 Feature Scaling	5
14 Parameter Trajectory	5
15 Momentum	6
16 Momentum Beta Experiment	6
17 Nesterov Accelerated Gradient	6
18 AdaGrad	6
19 RMSProp	7
20 Adam	7
21 Optimization Algorithm Comparison	7
22 Project Files	7
23 Commands	8
24 Repository Workflow	8
25 Final Formula Sheet	9
26 Experiment Checklist	9
27 Visualization Gallery from the Uploaded Plot Document	11

1 Project Overview

This project develops Linear Regression and a sequence of optimization methods from first principles. The central idea is to connect executable Python code with the underlying calculus, linear algebra, and numerical optimization.

Learning path: Linear Regression $\rightarrow$ MSE $\rightarrow$ Gradient $\rightarrow$ Gradient Descent $\rightarrow$ Learning Rate $\rightarrow$ Hessian and Stability $\rightarrow$ Feature Scaling $\rightarrow$ Momentum $\rightarrow$ Nesterov $\rightarrow$ AdaGrad $\rightarrow$ RMSProp $\rightarrow$ Adam

2 Dataset and Model

The working dataset is:

$$X = [1, 2, 3, 4, 5, 6], \quad y = [35, 42, 51, 58, 66, 73].$$

The single-feature linear regression model is

$$\hat{y}_i = b_0 + b_1 x_i$$

where b_0 is the intercept and b_1 is the slope.

Define

$$\boldsymbol{\theta} = \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}.$$

3 Mean Squared Error

The project uses Mean Squared Error (MSE) as the objective:

$$J(b_0, b_1) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Substituting the model,

$$J(b_0, b_1) = \frac{1}{n} \sum_{i=1}^n (b_0 + b_1 x_i - y_i)^2$$

The learning problem is therefore

$$\min_{b_0, b_1} J(b_0, b_1).$$

The residual can be written as

$$e_i = \hat{y}_i - y_i = b_0 + b_1 x_i - y_i.$$

Then

$$J = \frac{1}{n} \sum_{i=1}^n e_i^2.$$

4 Gradient: Full Derivation

The gradient is

$$\nabla J = \begin{bmatrix} \partial J / \partial b_0 \\ \partial J / \partial b_1 \end{bmatrix}$$

4.1 Derivative with respect to the intercept

Starting from

$$J = \frac{1}{n} \sum_i (b_0 + b_1 x_i - y_i)^2,$$

use the chain rule:

$$\begin{aligned} \frac{\partial J}{\partial b_0} &= \frac{1}{n} \sum_i 2(b_0 + b_1 x_i - y_i) \cdot 1 \\ &= \boxed{\frac{2}{n} \sum_i (\hat{y}_i - y_i)}. \end{aligned}$$

4.2 Derivative with respect to the slope

Similarly,

$$\begin{aligned} \frac{\partial J}{\partial b_1} &= \frac{1}{n} \sum_i 2(b_0 + b_1 x_i - y_i) x_i \\ &= \boxed{\frac{2}{n} \sum_i (\hat{y}_i - y_i) x_i}. \end{aligned}$$

Therefore,

$$\nabla J = \begin{bmatrix} \frac{2}{n} \sum_i (\hat{y}_i - y_i) \\ \frac{2}{n} \sum_i (\hat{y}_i - y_i) x_i \end{bmatrix}$$

5 Gradient Descent

The update rule is

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \nabla J(\boldsymbol{\theta}_t).$$

For the two parameters:

$$b_0^{(t+1)} = b_0^{(t)} - \alpha \frac{\partial J}{\partial b_0}$$

$$b_1^{(t+1)} = b_1^{(t)} - \alpha \frac{\partial J}{\partial b_1}.$$

The negative sign is essential because ∇J points toward local increase of the objective, while $-\nabla J$ points toward local decrease.

6 Loss Curve and Convergence

The loss history records

$$J_0, J_1, J_2, \dots, J_T.$$

A successful run shows a decreasing curve until it approaches a minimum. The project records the loss at each iteration so the optimization process can be inspected rather than hidden inside a library call.

7 Loss Surface and Geometry

Because there are two parameters, the objective can be viewed as

$$J = J(b_0, b_1).$$

A contour plot shows equal-loss curves, while the 3D plot displays the bowl-shaped surface directly.

The Gradient Descent path is the sequence

$$(b_0^{(0)}, b_1^{(0)}), (b_0^{(1)}, b_1^{(1)}), \dots, (b_0^{(T)}, b_1^{(T)}).$$

8 Gradient Direction and a First-Order Check

For a small parameter displacement $\Delta\theta$,

$$J(\theta + \Delta\theta) \approx J(\theta) + \nabla J(\theta)^T \Delta\theta.$$

Choosing

$$\Delta\theta = -\alpha \nabla J$$

gives

$$\begin{aligned} \Delta J &\approx -\alpha \nabla J^T \nabla J \\ &= -\alpha \|\nabla J\|^2 \leq 0. \end{aligned}$$

Thus, for sufficiently small positive α , the negative gradient is a local descent direction.

9 Gradient Checking

Analytical gradients can be verified with a central finite difference:

$$\boxed{\frac{\partial J}{\partial \theta} \approx \frac{J(\theta + \varepsilon) - J(\theta - \varepsilon)}{2\varepsilon}}$$

The implementation compares analytical and numerical values to catch derivative or coding errors.

10 Learning Rate

The learning rate α controls the update magnitude:

$$\Delta\theta = -\alpha \nabla J.$$

Small values lead to small steps and can be slow. Excessively large values can overshoot the minimum or become unstable.

11 Hessian, Eigenvalues, and Stability

The Hessian is

$$\boxed{H = \nabla^2 J(\theta)}.$$

For this two-parameter experiment,

$$\boxed{H = \begin{bmatrix} 2 & 7 \\ 7 & 30.3333 \end{bmatrix}}$$

with eigenvalues approximately

$$\lambda_1 \approx 0.36494, \quad \lambda_2 \approx 31.96839.$$

Hence

$$\lambda_{\max} \approx 31.96839.$$

For this quadratic objective and the stated gradient convention, the stability condition is

$$0 < \alpha < \frac{2}{\lambda_{\max}}.$$

Numerically,

$$\alpha < 0.06256 \text{ (approximately)}.$$

12 Stability Experiments

The experiments tested learning rates well below and above the stability boundary. The observed behavior is consistent with the theory: below the boundary the loss can decrease, while values above it can make the recurrence unstable and cause rapid numerical growth.

For the first-30-iteration experiment, very large learning rates produce enormous loss values, so a logarithmic scale is useful for visualizing both stable and unstable cases.

13 Feature Scaling

Standardization uses

$$x' = \frac{x - \mu}{\sigma}.$$

For the given data, the mean is $\mu = 3.5$ and the standardized values are approximately

$$[-1.464, -0.878, -0.293, 0.293, 0.878, 1.464].$$

For the scaled feature,

$$H' \approx \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix},$$

so the eigenvalues become approximately 2 and 2. The condition number is

$$\kappa(H) = \frac{\lambda_{\max}}{\lambda_{\min}}.$$

For the original coordinates,

$$\kappa(H) \approx \frac{31.96839}{0.36494} \approx 87.6,$$

whereas for the standardized feature,

$$\kappa(H') = 1.$$

This explains the more balanced optimization geometry after scaling.

14 Parameter Trajectory

The parameter path makes the geometry visible directly. In ill-conditioned coordinates, the loss surface has different curvature scales in different directions, so Gradient Descent may take an inefficient, curved or zig-zagging path. After scaling, the coordinates are much better balanced.

15 Momentum

Momentum introduces a velocity variable:

$$v_t = \beta v_{t-1} - \alpha \nabla J(\theta_t)$$

followed by

$$\theta_{t+1} = \theta_t + v_t.$$

If $\beta = 0$, the method reduces to ordinary Gradient Descent.

The expanded recurrence shows that current velocity contains a decaying memory of previous gradients. Thus momentum can build speed along persistent directions and reduce some forms of slow progress in elongated valleys, but excessive momentum can also lead to overshooting and oscillation.

16 Momentum Beta Experiment

The coefficient β controls how much previous velocity is retained. The experiments included

$$\beta \in \{0, 0.5, 0.9, 0.99\}.$$

As β increases, more history is preserved. The experiment shows increasingly visible oscillation at high momentum, especially for $\beta = 0.99$ on the chosen configuration.

17 Nesterov Accelerated Gradient

Nesterov changes where the gradient is evaluated. First compute a look-ahead point:

$$\theta_{\text{lookahead}} = \theta_t + \beta v_t.$$

Then evaluate

$$g_t = \nabla J(\theta_{\text{lookahead}})$$

and update

$$v_{t+1} = \beta v_t - \alpha g_t$$

$$\theta_{t+1} = \theta_t + v_{t+1}.$$

The key distinction is therefore:

Momentum uses the current-position gradient; Nesterov uses the look-ahead gradient.

18 AdaGrad

AdaGrad accumulates squared gradients:

$$G_t = G_{t-1} + g_t^2.$$

The coordinate-wise update is

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{G_t} + \epsilon} g_t.$$

The effective learning rate decreases as the accumulator grows. A key limitation is that the accumulator never forgets past squared gradients, so the effective learning rate can eventually become very small.

19 RMSProp

RMSProp replaces permanent accumulation with an exponential moving average:

$$S_t = \beta S_{t-1} + (1 - \beta) g_t^2.$$

The update is

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{S_t} + \epsilon} g_t.$$

Old gradients gradually lose influence, avoiding AdaGrad's continuously shrinking accumulator.

20 Adam

Adam combines a first moment and a second moment.

First moment:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Second moment:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

Final update:

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Typical hyperparameter values used in practice include

$$\beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 10^{-8}.$$

21 Optimization Algorithm Comparison

Algorithm	Core mechanism	Main state
Gradient Descent	Current gradient	None
Momentum	Gradient + accumulated velocity	First-order history
Nesterov	Look-ahead gradient + momentum	Position + velocity
AdaGrad	Per-parameter accumulated squared gradients	Sum of squared gradients
RMSProp	Exponential average of squared gradients	Recent squared-gradient history
Adam	First + second moments + bias correction	Gradient and squared-gradient statistics

22 Project Files

File	Purpose
<code>linear_regression.py</code>	Baseline linear regression, MSE, Gradient Descent, predictions, plots.

loss_surface.py	2D MSE contour plot and Gradient Descent parameter path.
loss_surface_3d.py	3D loss surface and optimization path.
gradient_vector.py	Gradient and negative-gradient visualization.
gradient_check.py	Analytical versus finite-difference gradient check.
learning_rate_experiment.py	Learning-rate comparison and convergence behavior.
hessian_analysis.py	Hessian, eigenvalues, and theoretical stability limit.
stability_boundary.py	Experiments close to the theoretical stability boundary.
feature_scaling.py	Original versus standardized feature geometry and convergence.
parameter_trajectory.py	Parameter-space paths and optimum comparison.
momentum_gradient_descent.py	Vanilla GD versus Momentum.
momentum_beta_experiment.py	Effect of β on Momentum behavior.
nesterov_momentum.py	Vanilla GD versus Momentum versus Nesterov.
adagrad.py	AdaGrad implementation and loss visualization.
rmsprop.py	RMSProp implementation and loss visualization.
adam.py	Adam implementation and loss visualization.
README.md	GitHub project documentation with rendered mathematics.
plots/	Visualization gallery from the experiments.
docs/	Detailed report/documentation artifacts.

23 Commands

The project can be run from PowerShell after changing into the project directory:

```
cd C:\\Users\\Lenovo\\machine-learning-algorithms\\01_linear_regression
python linear_regression.py
python loss_surface.py
python loss_surface_3d.py
python gradient_vector.py
python gradient_check.py
python learning_rate_experiment.py
python hessian_analysis.py
python stability_boundary.py
python feature_scaling.py
python parameter_trajectory.py
python momentum_gradient_descent.py
python momentum_beta_experiment.py
python nesterov_momentum.py
python adagrad.py
python rmsprop.py
python adam.py
```

Dependencies:

```
pip install numpy matplotlib
```

24 Repository Workflow

For the Git repository:

```
git status
git add .
git commit -m "Add linear regression and optimization from scratch"
git branch -M main
git remote -v
git push -u origin main
```

The project is intended to remain isolated as its own Git repository under `C:`

Users

Lenovo

machine-learning-algorithms.

25 Final Formula Sheet

Model: $\hat{y} = b_0 + b_1x$

MSE: $J = \frac{1}{n} \sum (\hat{y} - y)^2$

Gradient: $\nabla J = \begin{bmatrix} \partial J / \partial b_0 \\ \partial J / \partial b_1 \end{bmatrix}$

GD: $\theta_{t+1} = \theta_t - \alpha \nabla J_t$

Finite difference: $\frac{\partial J}{\partial \theta} \approx \frac{J(\theta + \epsilon) - J(\theta - \epsilon)}{2\epsilon}$

Hessian: $H = \nabla^2 J$

Stability: $0 < \alpha < 2/\lambda_{\max}$

Scaling: $x' = (x - \mu)/\sigma$

Momentum: $v_t = \beta v_{t-1} - \alpha \nabla J_t, \quad \theta_{t+1} = \theta_t + v_t$

Nesterov: $\theta_{\text{lookahead}} = \theta_t + \beta v_t$

AdaGrad: $G_t = G_{t-1} + g_t^2$

RMSPProp: $S_t = \beta S_{t-1} + (1 - \beta) g_t^2$

Adam: $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$

$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$

$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$

26 Experiment Checklist

- Linear Regression model and MSE implemented from scratch.
- Analytical gradient derived and implemented.
- Loss versus iteration visualized.
- 2D contour and 3D loss surface visualized.
- Gradient and negative-gradient directions verified numerically.
- Learning-rate stability explored experimentally.
- Hessian and eigenvalues computed.

- Stability boundary calculated and tested.
- Feature scaling and condition number examined.
- Momentum and its β coefficient studied.
- Nesterov look-ahead update implemented.
- AdaGrad, RMSProp, and Adam implemented from scratch.

27 Visualization Gallery from the Uploaded Plot Document

The following pages reproduce the complete 13-page visualization set supplied for this project. The source plot document contains the 3D loss surface, AdaGrad, Adam, feature scaling, gradient direction, first-30-iteration learning-rate experiment, Gradient Descent learning, stability boundary, parameter trajectory, Momentum, Momentum beta, RMSProp, and the Vanilla/Momentum/Nesterov comparison.

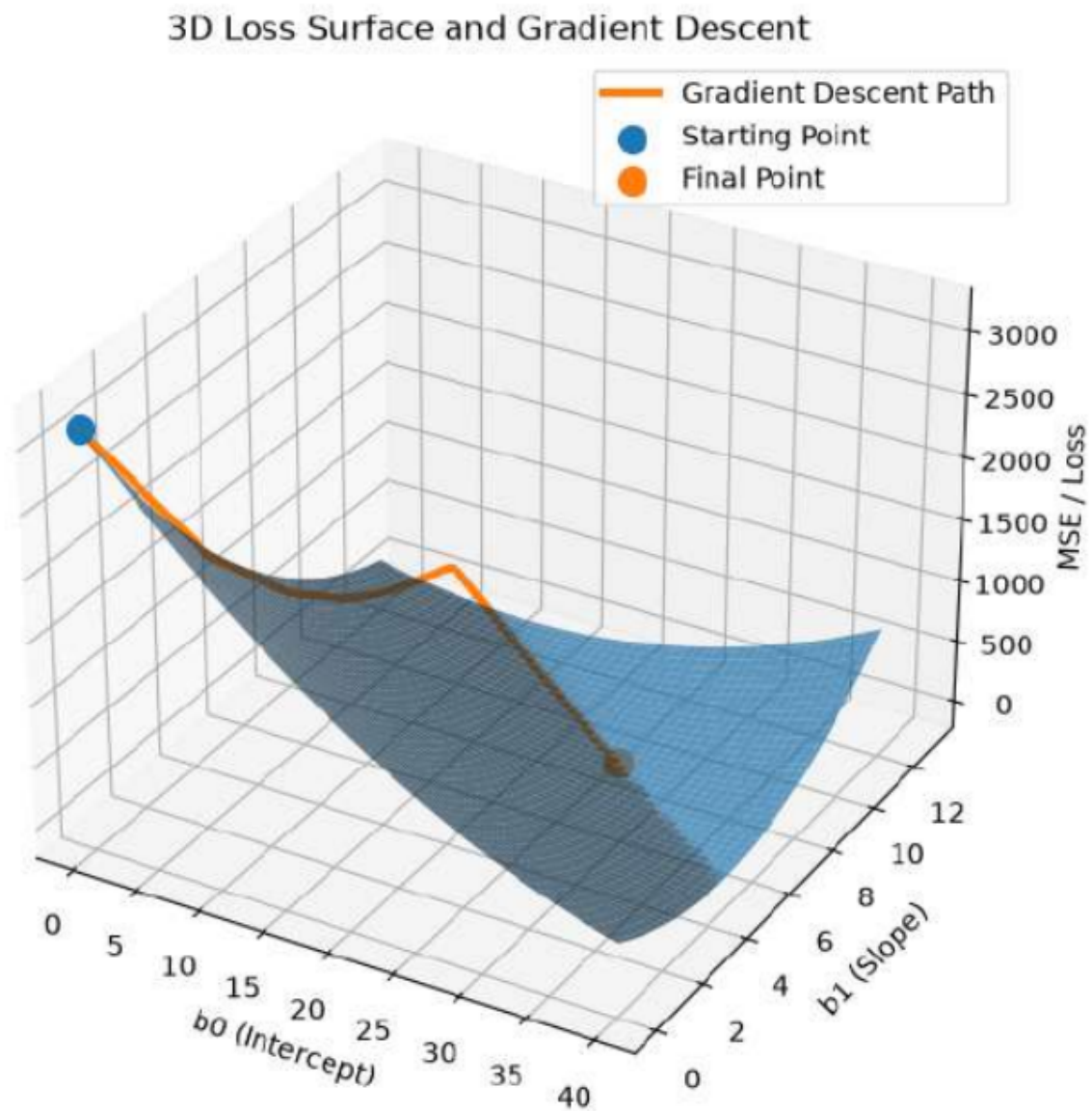


Figure 1: 3D Loss Surface and Gradient Descent.

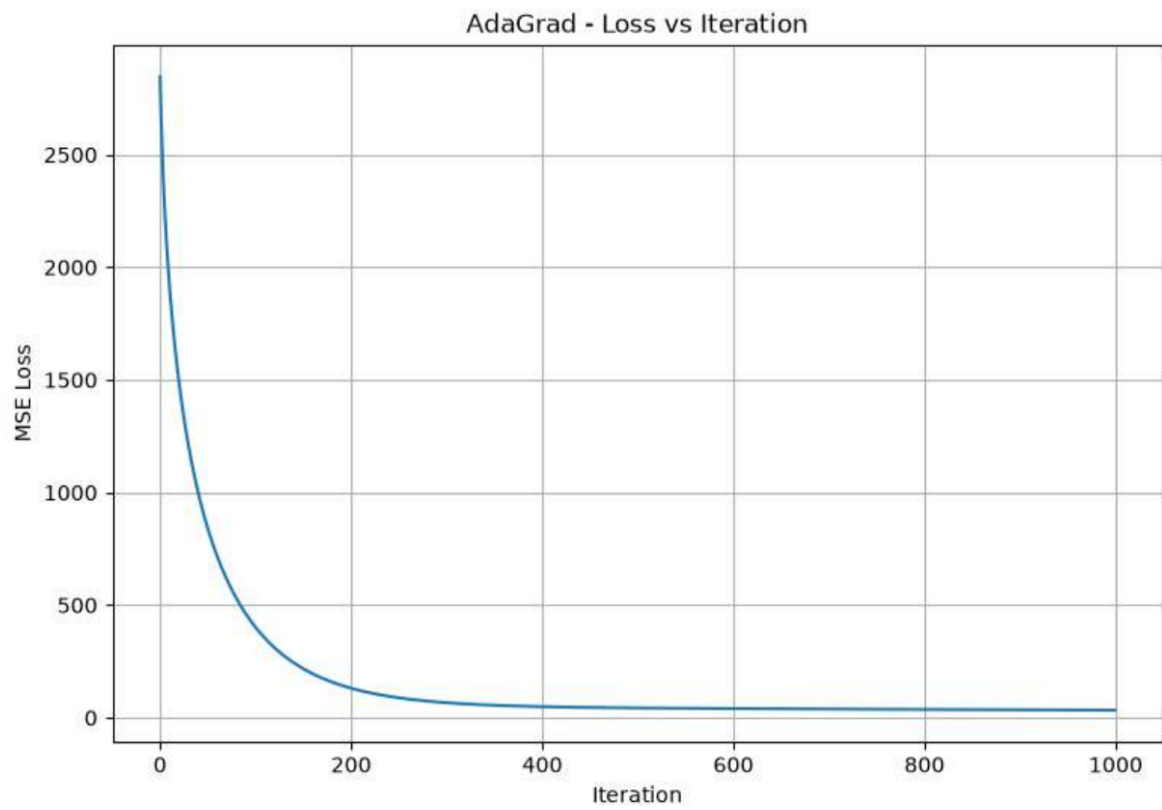


Figure 2: AdaGrad - Loss vs Iteration.

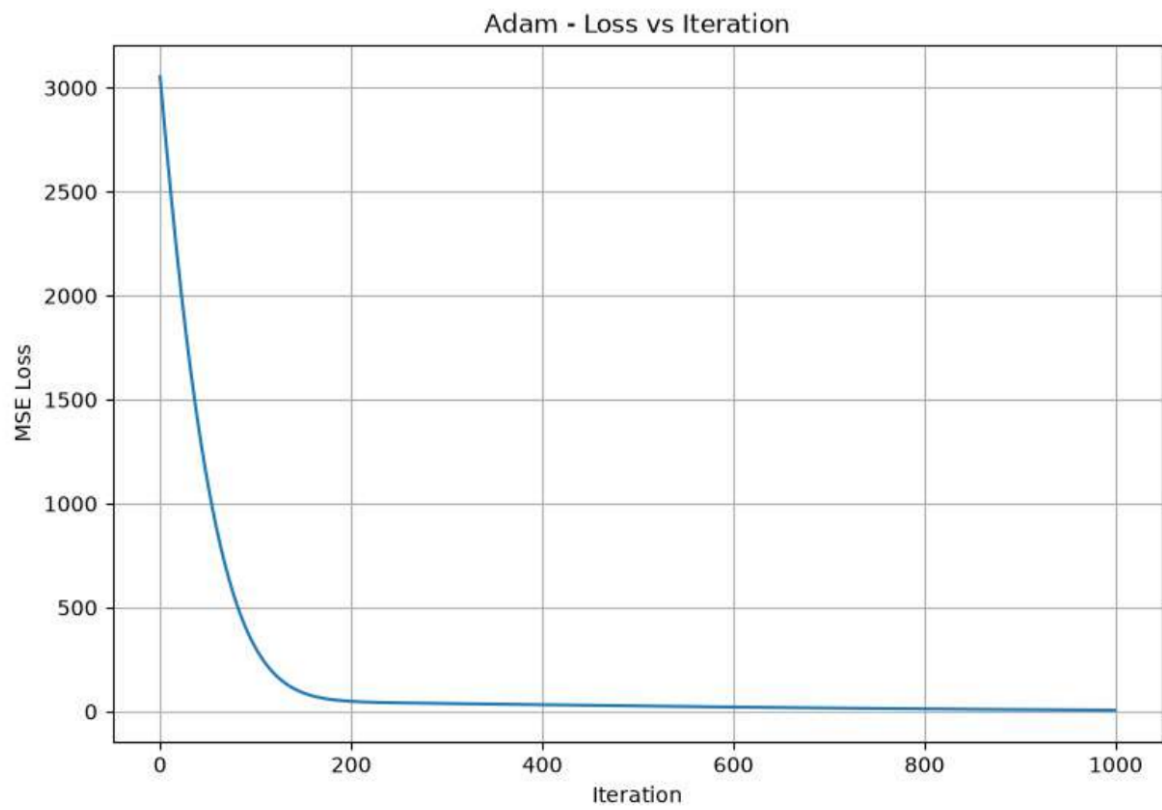


Figure 3: Adam - Loss vs Iteration.

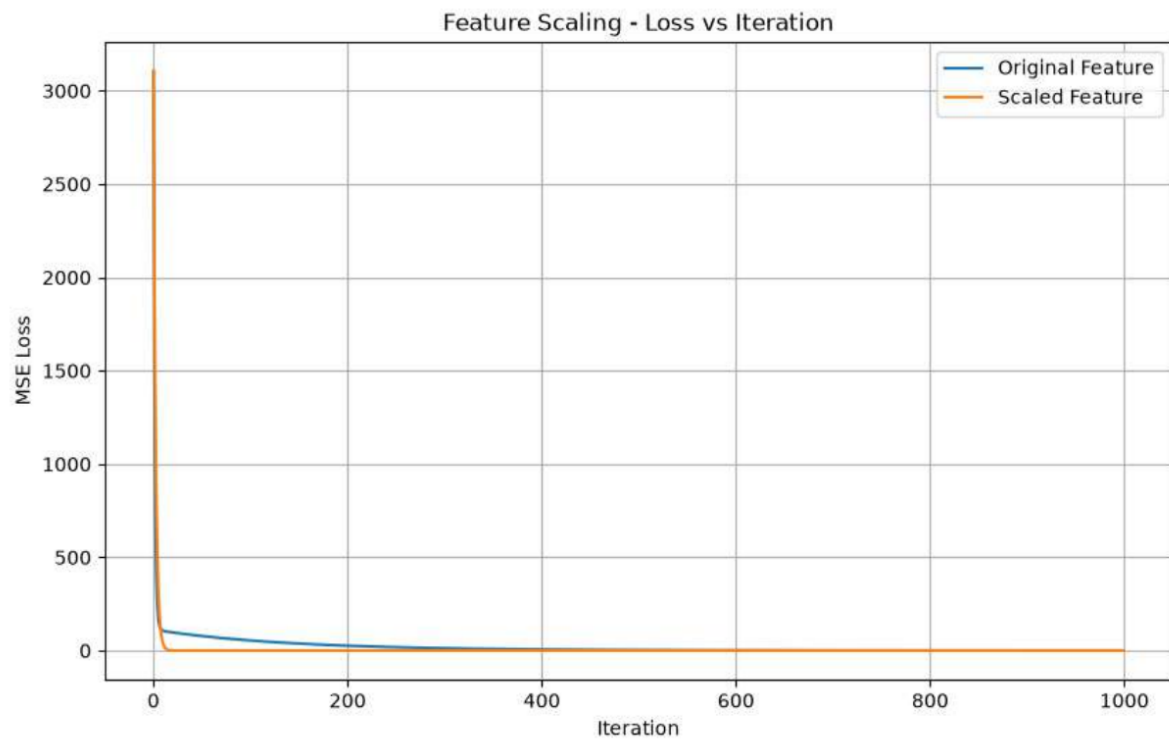


Figure 4: Feature Scaling - Loss vs Iteration.

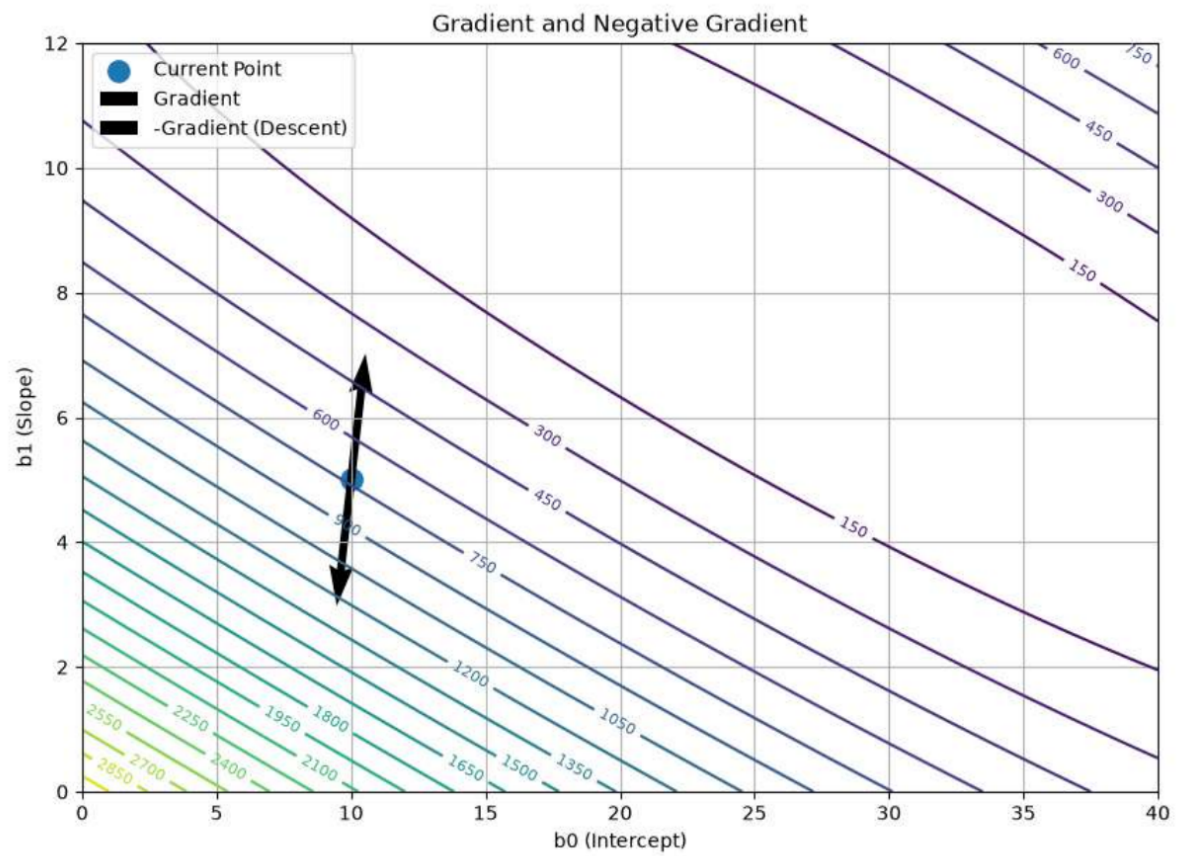


Figure 5: Gradient and Negative Gradient.

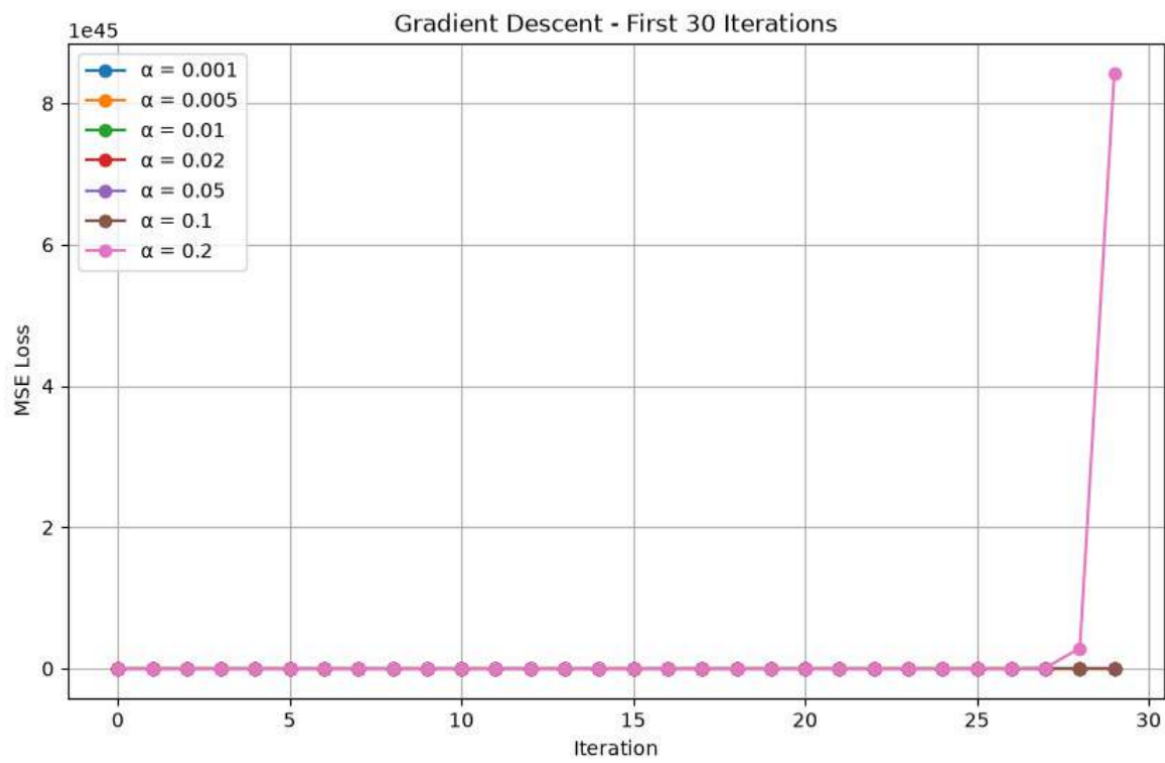


Figure 6: Gradient Descent - First 30 Iterations.

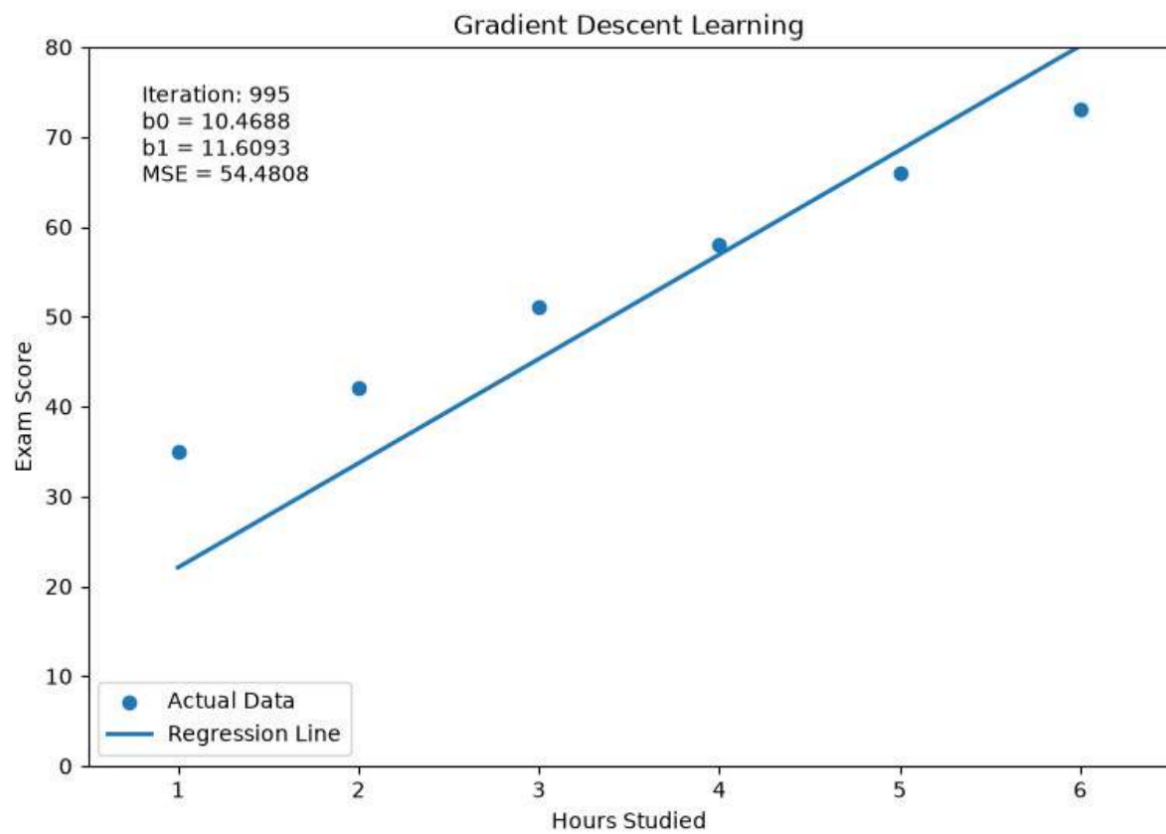


Figure 7: Gradient Descent Learning.

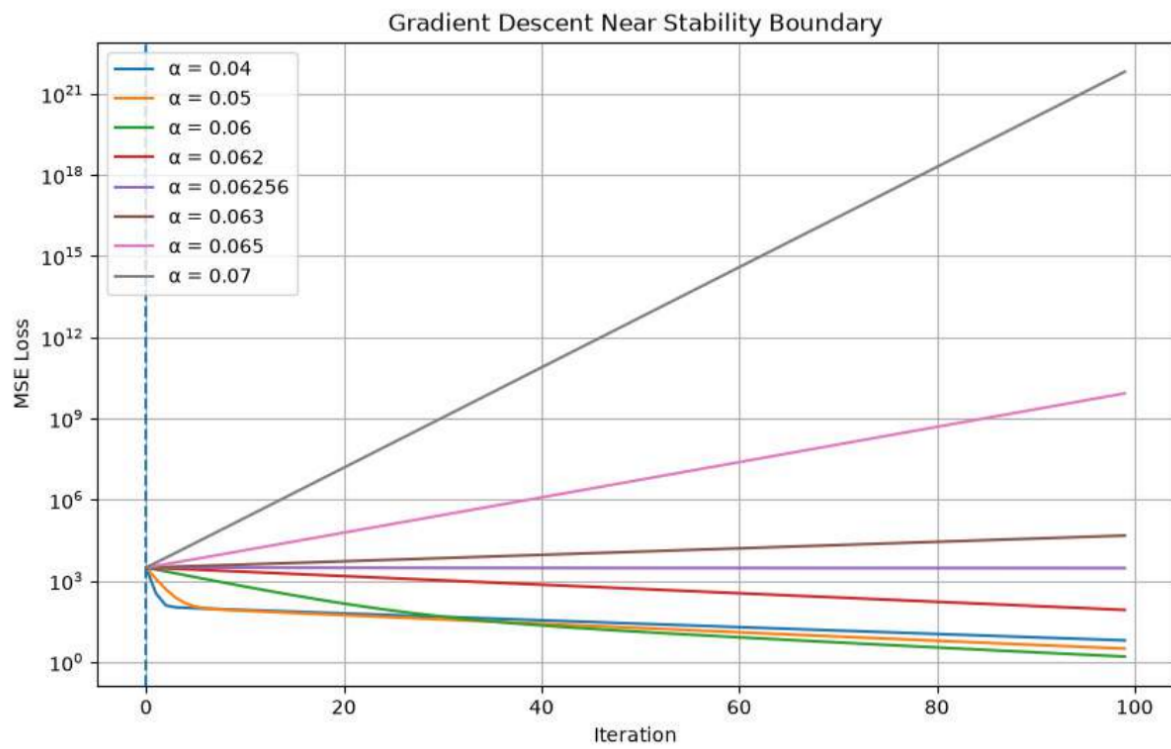


Figure 8: Gradient Descent Near Stability Boundary.

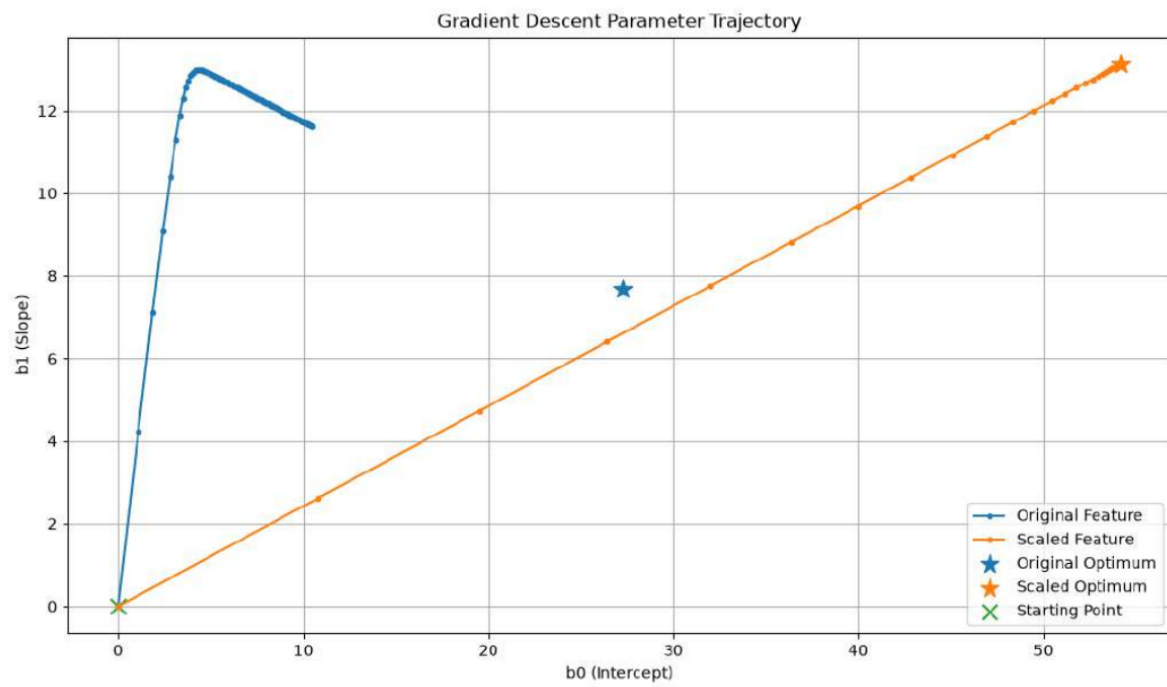


Figure 9: Gradient Descent Parameter Trajectory.

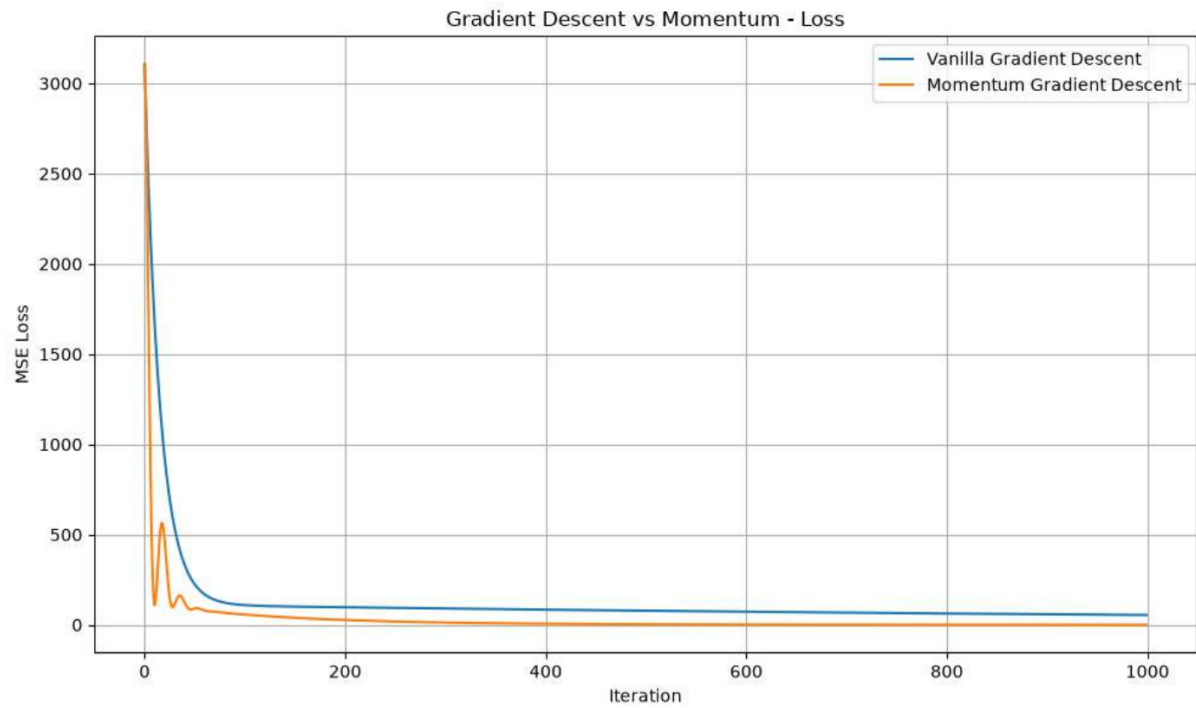


Figure 10: Gradient Descent vs Momentum - Loss.

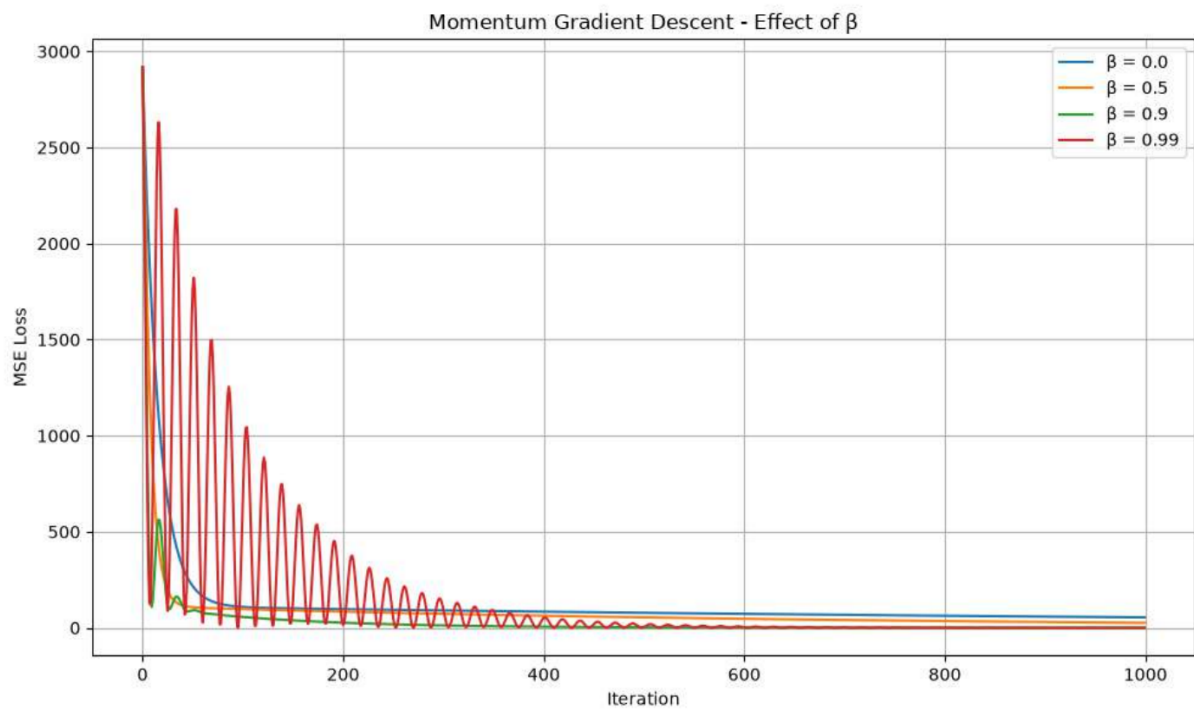


Figure 11: Momentum Gradient Descent - Effect of β .

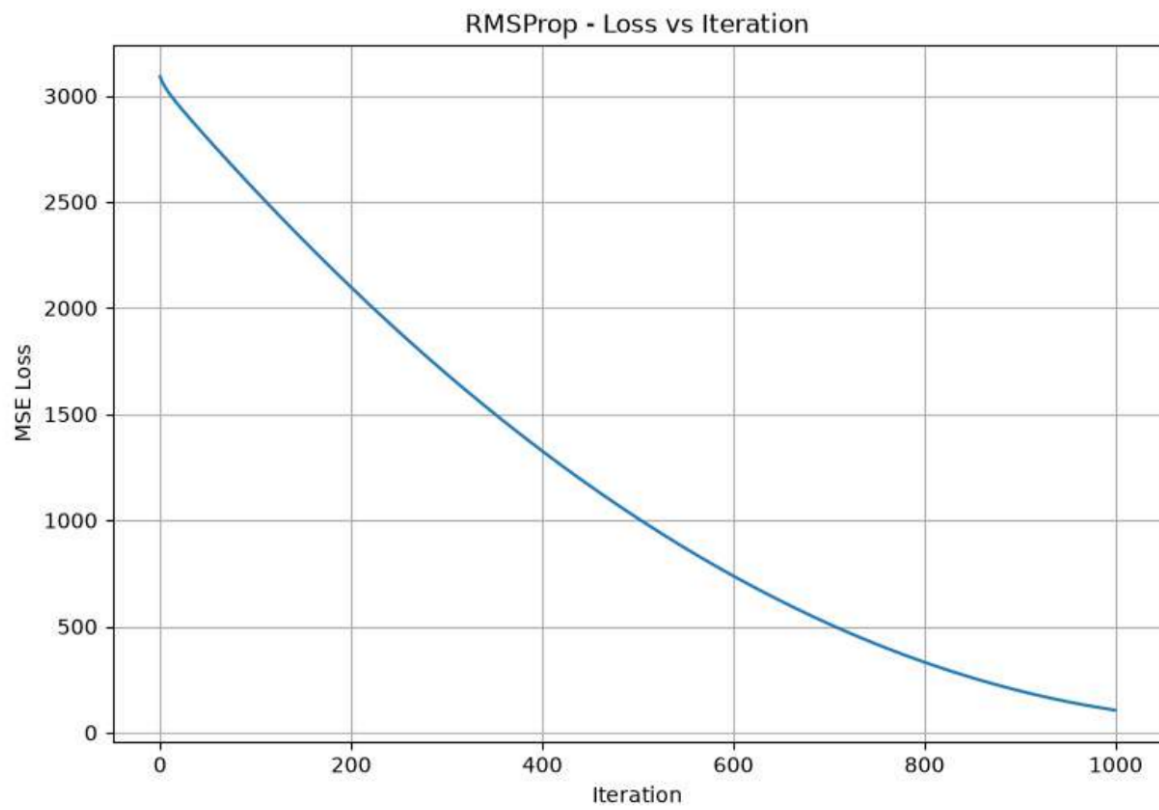


Figure 12: RMSProp - Loss vs Iteration.

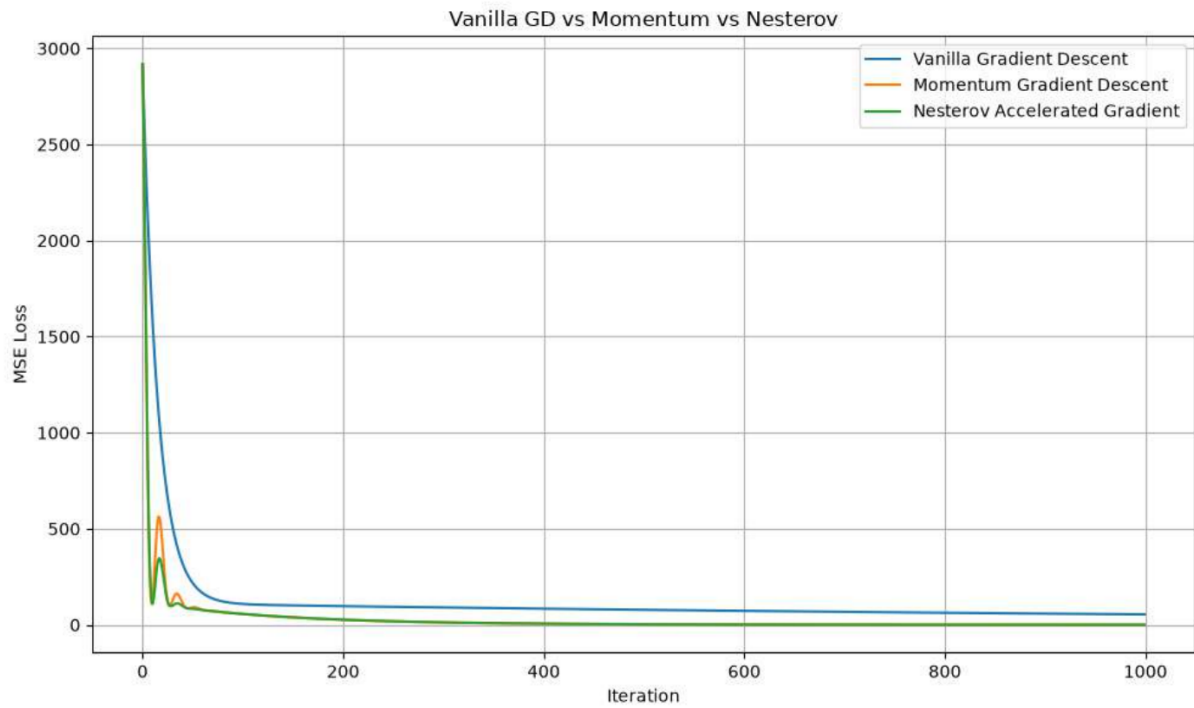


Figure 13: Vanilla GD vs Momentum vs Nesterov.